

SnapKV, SpecKV and SpecPC

Dec 1st, 2025

SnapKV

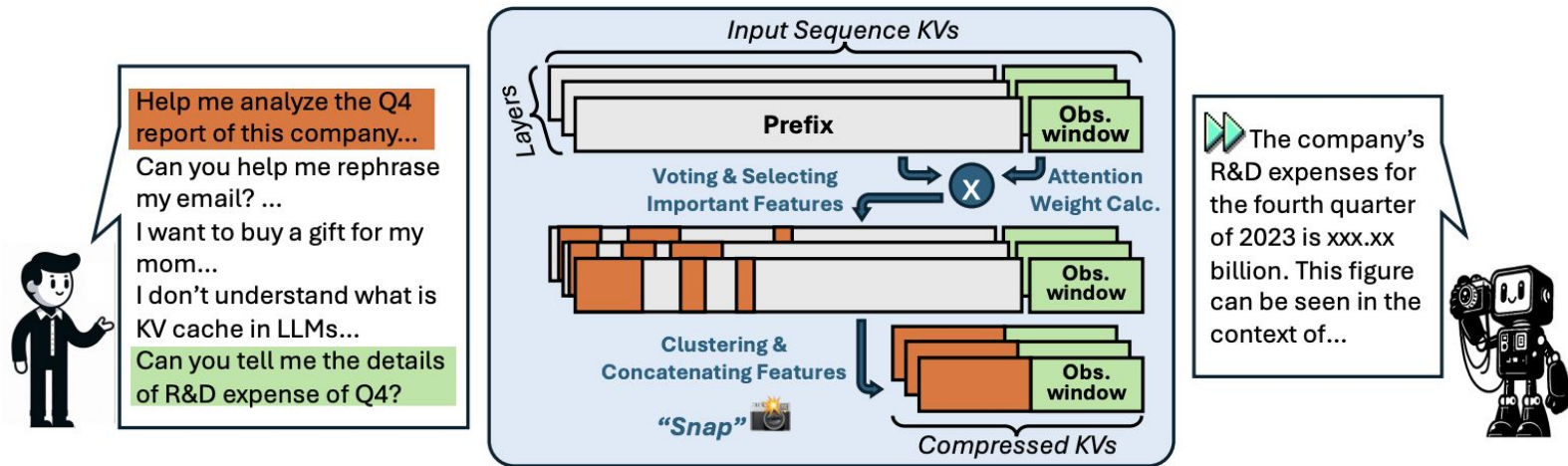


Figure 1: The graph shows the simplified workflow of SnapKV, where the orange area represents the cluster of features per head selected by SnapKV. These features are then used to form new Key-Value pairs concatenated with the features in the observation window. Together, the selected prefix and observation windows constitute the new KV cache utilized for the generation.

SnapKV

- “Only a portion of prompt tokens convey essential information for response generation, and these tokens remain unchanged during generation.”
- Those tokens can be found before generation.

SnapKV

- **Prompt Length** (L_{prompt}): The total length of the user-provided input.
- **Observation Window** (L_{obs}): The last segment of the prompt. This window is crucial for analyzing the influence of different contexts on attention allocation patterns.
- **Prefix Length** (L_{prefix}): The length of the input preceding the observation window. It is part of the prompt and does not include the observation window. Overall, we have:

$$L_{\text{prompt}} = L_{\text{prefix}} + L_{\text{obs}} \quad (1)$$

SnapKV

- **Voting:** The process of calculating attention weights for each query within the observation window across all heads, aggregating these weights to highlight the prefix positions that are considered most significant. For a single batch of sequence, formally:

$$\mathbf{C} = \sum_{i=0}^{L_{\text{obs}}} \mathbf{W}_{\text{obs}}[:, i, :] \quad (2)$$

$$I = \text{Top}_k(\mathbf{C}, k) \quad (3)$$

where $\text{Top}_k(\mathbf{C}, k)$ selects the indices I of the top k values in tensor $\mathbf{C}$ per head. k is defined as $\lfloor p \times L_{\text{prefix}} \rfloor$, where p stands for the compression rate. The tensor $\mathbf{W}_{\text{obs}} \in \mathbb{R}^{N \times L_{\text{obs}} \times L_{\text{prefix}}}$ represents the subset of the prompt softmax-normalized attention features over N heads.

SnapKV

- **Hit Rate:** We define attention features above a predefined threshold θ during generation as *important* features. The hit rate, H , is the number of important features successfully selected by the previous voting process over the total number of important features. H quantifies the effectiveness of the voting mechanism and is calculated as follows:

$$\mathbf{M}_{\text{vote_obs}} = \text{zeros_like}(\mathbf{A}_{\text{cur}}) \quad (4)$$

$$\mathbf{M}_{\text{vote_obs}}[I] = 1 \quad (5)$$

$$\mathbf{M}_{\text{threshold_cur}} = \mathbf{1}(\mathbf{A}_{\text{cur}} > \theta) \quad (6)$$

$$\mathbf{O} = \mathbf{M}_{\text{threshold_cur}} \wedge \mathbf{M}_{\text{vote_obs}} \quad (7)$$

$$H = \frac{\sum \mathbf{O}}{\sum \mathbf{M}_{\text{threshold_cur}}} \quad (8)$$

$$\mathbf{A}_{\text{cur}} \in \mathbb{R}^{N \times L_{\text{prefix}}}$$

SnapKV

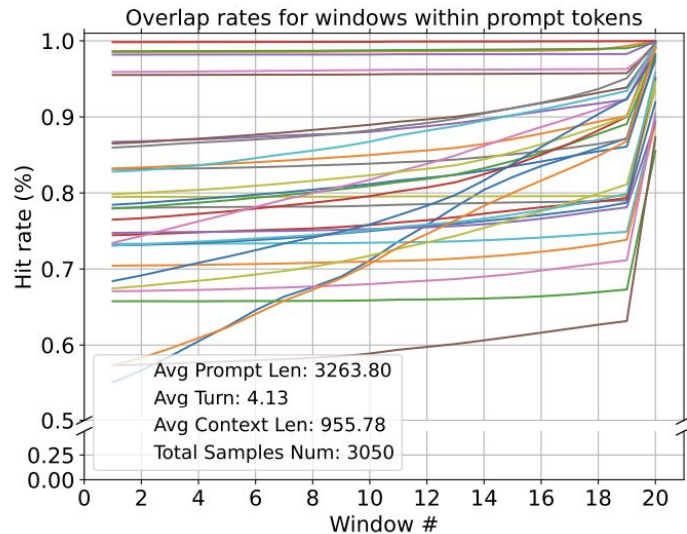


Figure 2: The overlap rates between attention features of the input sequence, selected by various windows along the input and during generation, with each line representing a model layer.

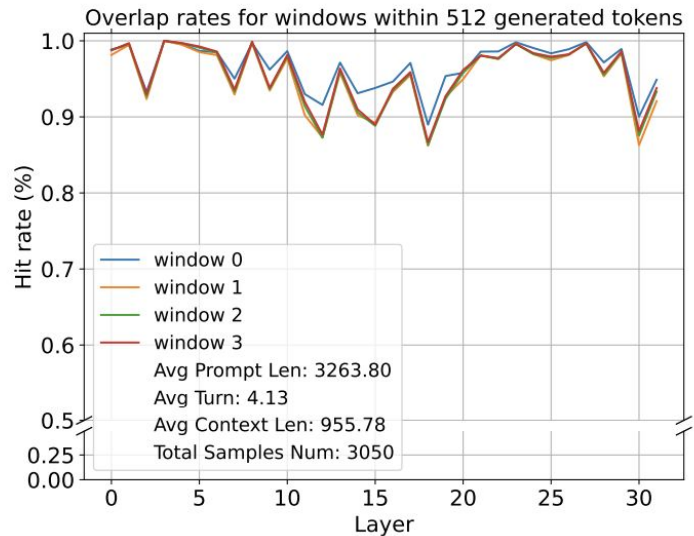


Figure 3: The layer-wise overlap rates between input sequence attention features selected by the last window of input sequence and those selected by 4 windows along generation.

SnapKV

```
1 def snap_kv(query_states, key_states, value_states, window_size, max_capacity_prompt,
   kernel_size):
2     bsz, num_heads, q_len, head_dim = query_states.shape
3     # Ensure it is the prompt phase.
4     assert key_states.shape[-2] == query_states.shape[-2]
5     if q_len < max_capacity_prompt:
6         return key_states, value_states
7     else:
8         # Compute attention weights of observing window's queries and prefix context's Keys.
9         attn_weights = compute_attn(query_states[..., -window_size:, :], key_states,
   attention_mask)
10        # Sum the weight along the query dimension.
11        vote = attn_weights[..., -window_size:, :-window_size].sum(dim=-2)
12        # Apply 1D pooling for clustering.
13        pool_vote = pool1d(vote, kernel_size=kernel_size, padding=kernel_size//2, stride=1)
14        # Select top-k indices based on the pooled weights to identify important positions.
15        indices = pool_vote.topk(max_capacity_prompt - window_size, dim=-1).indices
16        # Expand the indices to match the head dimension for gathering.
17        indices = indices.unsqueeze(-1).expand(-1, -1, -1, head_dim)
18        # Gather the compressed past key and value states based on the selected indices.
19        k_past_compress = key_states[..., :-window_size, :].gather(dim=2, index=indices)
20        v_past_compress = value_states[..., :-window_size, :].gather(dim=2, index=indices)
21        k_obs = key_states[..., -window_size:, :]
22        v_obs = value_states[..., -window_size:, :]
23        key_states = torch.cat([k_past_compress, k_obs], dim=2)
24        value_states = torch.cat([v_past_compress, v_obs], dim=2)
25        return key_states, value_states
```

Listing 1: Implementation of SnapKV in pseudo PyTorch style.

SnapKV

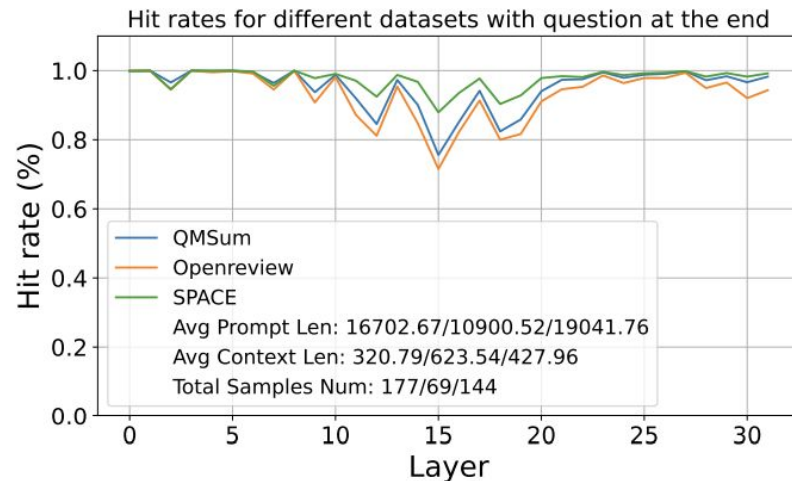
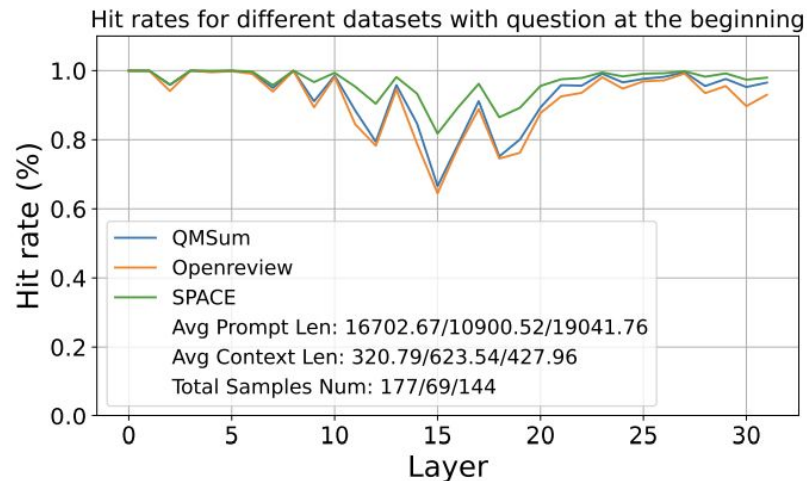


Figure 5: The layer-wise average hit rate of important positions used by prompts with questions at the beginning and the end.

SnapKV

Table 1: Performance comparison of SnapKV and H2O across various LLMs on LongBench.

	LLMs *	Single-Document QA			Multi-Document QA			Summarization			Few-shot Learning			Synthetic		Code	
		NrrvQA	Qasper	MF-en	HotpotQA	2WikiMQA	Musique	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PRe	Lcc	RB-P
LWMChat	All KV	18.18	25.56	40.94	24.57	19.39	10.49	27.97	24.9	24.81	71.0	60.9	39.73	3.17	3.5	44.4	43.82
	SnapKV: 1024	18.02	23.73	40.25	24.61	19.84	10.77	19.79	24.44	23.53	70.0	61.42	39.64	1.67	3.0	43.34	44.0
	SnapKV: 2048	17.92	25.03	41.38	24.49	19.38	11.34	21.6	24.22	24.36	70.0	61.11	39.91	2.17	4.0	44.46	44.92
	SnapKV: 4096	17.92	25.47	40.76	24.92	19.53	11.27	25.34	25.42	24.58	70.5	61.08	39.62	3.17	4.0	44.49	44.08
	H2O: 4096	13.17	24.82	20.01	16.86	9.74	7.2	25.77	23.26	23.83	71.0	61.06	40.33	0.0	0.0	41.52	40.97
LongChat	All KV	20.88	29.36	43.2	33.05	24.58	14.66	30.89	22.76	26.61	66.5	83.99	40.83	0.0	30.5	54.89	59.05
	SnapKV: 1024	19.32	26.6	37.93	34.15	23.34	12.71	23.45	21.81	24.93	65.0	80.88	38.19	0.0	31.0	53.63	57.62
	SnapKV: 2048	19.28	28.81	40.26	35.31	23.75	13.44	26.3	22.29	25.73	66.0	79.93	39.59	0.0	31.0	56.05	58.61
	SnapKV: 4096	20.68	29.34	42.21	33.95	24.88	14.15	28.55	23.11	26.45	66.0	81.25	40.52	0.0	29.5	54.79	58.81
	H2O: 4096	19.31	28.3	37.75	30.51	23.06	11.76	27.55	21.37	26.49	66.0	75.8	39.92	0.0	25.5	53.56	55.53
Mistral	All KV	26.82	33.06	49.28	42.77	27.33	19.27	32.85	24.25	27.06	71.0	86.23	42.98	2.75	86.98	55.51	52.88
	SnapKV: 1024	25.54	29.51	49.25	40.94	25.7	19.42	25.89	23.82	26.11	69.5	86.48	42.06	2.98	88.56	55.65	51.87
	SnapKV: 2048	25.89	32.47	48.6	41.71	27.31	18.69	28.81	24.5	26.6	70.0	86.27	42.47	3.09	87.43	55.93	52.01
	SnapKV: 4096	26.41	33.36	49.81	42.32	27.93	18.76	30.74	24.19	27.08	71.0	86.25	43.01	2.73	86.18	55.62	52.65
	H2O: 4096	22.61	29.06	47.22	36.54	20.6	16.25	30.0	23.8	26.75	70.5	86.16	42.97	3.46	86.38	53.72	51.1
Mixtral	All KV	26.81	37.06	51.55	47.77	32.46	26.59	34.25	26.05	27.91	76.0	90.57	46.98	5.5	100.0	69.07	69.65
	SnapKV: 1024	26.01	34.65	51.58	48.23	32.67	25.92	27.77	25.0	27.25	74.5	90.42	46.48	5.5	99.5	69.02	68.98
	SnapKV: 2048	27.12	36.9	51.91	47.46	33.23	26.27	30.19	25.84	27.8	76.0	90.24	46.31	5.5	100.0	68.72	70.01
	SnapKV: 4096	26.46	37.03	52.62	47.71	33.35	26.45	32.64	25.87	27.94	75.5	90.71	47.14	5.5	100.0	68.81	69.56
	H2O: 4096	20.45	32.09	48.02	34.76	25.69	16.5	29.76	23.53	26.84	74.5	90.24	47.1	7.06	99.42	64.91	63.52

* Credit to Jin et al. [20] for the template used in the table.

Draft-based Approximate Inference Framework

- “Token importance estimation can be improved by incorporating information from future tokens.”
- “Draft-based Approximate Inference, a lookahead-based framework that uses a smaller, more efficient draft model to approximate future outputs with minimal overhead”

Draft-based Approximate Inference Framework

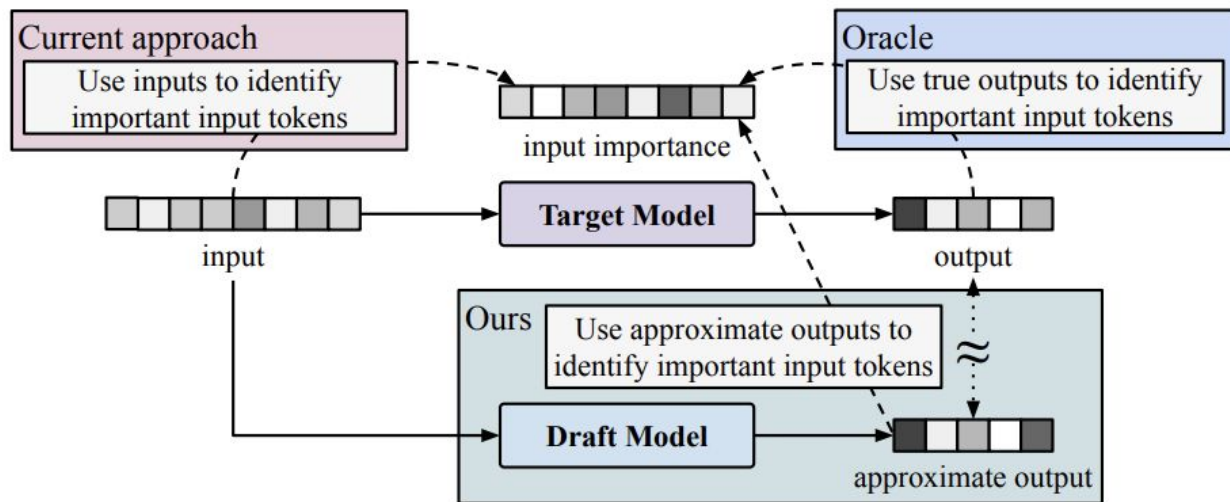


Figure 1: Overview of our Draft-based Approximate Inference framework for input token importance estimation. Unlike prior methods that rely only on input tokens, our approach incorporates draft model predictions of future output tokens, yielding more accurate importance estimates. This better aligns with the hypothetical oracle setting, where the true output is known and influential tokens can be precisely identified.

SpecKV

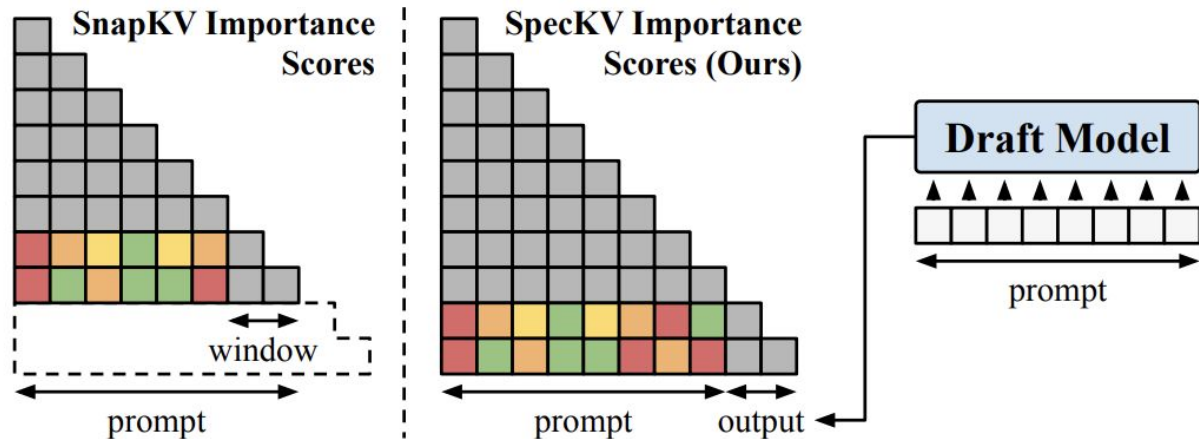


Figure 2: Overview of SpecKV: Instead of using only the last prompt tokens like SnapKV, SpecKV employs a lightweight draft model to generate lookahead tokens, providing richer context for more accurate KV importance estimation.

SpecKV

To guide both sparse prefilling and KV cache dropping, SpecKV estimates the importance of each KV pair in the input sequence. We define importance as the average attention activation from output queries to input keys. Specifically, the vector of importance scores and its approximation are given by

$$s^T = \frac{1}{n_{\text{out}}} \sum_{i=1}^{n_{\text{out}}} \text{Softmax} \left(\frac{x_i^{(o)T} W_q W_k^T X^T}{\sqrt{d}} \right) \quad \text{and} \quad \hat{s}^T = \frac{1}{n_{\text{out}}} \sum_{i=1}^{n_{\text{out}}} \text{Softmax} \left(\frac{\hat{x}_i^{(o)T} W_q W_k^T X^T}{\sqrt{d}} \right), \quad (1)$$

where $X = [x_1, \dots, x_{n_{\text{in}}}]^T \in \mathbb{R}^{n_{\text{in}} \times d}$ is the matrix of input embeddings, $x_i^{(o)} \in \mathbb{R}^d$ is the i th output embedding, and $\hat{x}_i^{(o)} \in \mathbb{R}^d$ is the i th approximate output embedding (from the draft model). s_i and $\hat{s}_i$ denote the importance of the i th KV pair.

Theorem 1. *If $\|x_i^{(o)} - \hat{x}_i^{(o)}\|_2 \leq \epsilon$ for all i and $\|x_j\|_2 \leq \sqrt{d}$ for all j , then $\|s - \hat{s}\|_2 \leq \epsilon \|W_q W_k^T\|_2^2$.*

This result shows that for a single attention layer, the worst-case error in the approximate importance scores is proportional to the worst-case error in the approximate output embeddings, implying that SpecKV provides reliable estimates as long as the draft model remains reasonably accurate (see Appendix B.1 for proof).

Algorithm 1 SpecKV

1: Input:

Input sequence x with length n_{in}

Parameters: Number of lookahead tokens $n_{\text{lookahead}}$, Maximum cache capacity C_{max} ,

Compression window size n_{window} , Kernel size k , Prefill window size n_{slash} ,

Number of global tokens in prefill n_{vert}

2: Generate a draft output y_{draft} of length $n_{\text{lookahead}}$ using the draft model.

3: Forward x and y_{draft} through the target model.

4: **for** each attention head in target model **do**

5: $X \leftarrow$ Target model hidden states from prompt at current layer

$\triangleright X \in \mathbb{R}^{n_{\text{in}} \times d}$

6: $Y \leftarrow$ Target model hidden states from draft output at current layer

$\triangleright Y \in \mathbb{R}^{n_{\text{lookahead}} \times d}$

7: $m \leftarrow n_{\text{in}} - n_{\text{window}}$

8: $A \leftarrow \text{CrossAttention}(Q \text{ from } [X_Y^m], K/V \text{ from } X_{:m})$

$\triangleright$ Compute attention

9: $s \leftarrow \text{MaxReduce}(A)$

$\triangleright \mathbb{R}^{n_1 \times n_2} \rightarrow \mathbb{R}^{n_2}$

10: $s \leftarrow \text{AvgPool1D}(s, k)$

$\triangleright$ Smooth attention

11: $i_{\text{vert}} \leftarrow \text{topk}(s, n_{\text{vert}})$

$\triangleright$ Select global tokens (Sparse prefill)

12: $i_{\text{slash}} \leftarrow \{1, 2, \dots, n_{\text{slash}}\}$

$\triangleright$ Sliding window (Sparse prefill)

13: $i_{\text{cache}} \leftarrow \text{topk}(s, C_{\text{max}} - n_{\text{window}}) \cup \{m + 1, m + 2, \dots, n_{\text{in}}\}$

$\triangleright$ Select KVs

14: output $\leftarrow \text{VerticalSlash}(X, i_{\text{vert}}, i_{\text{slash}})$

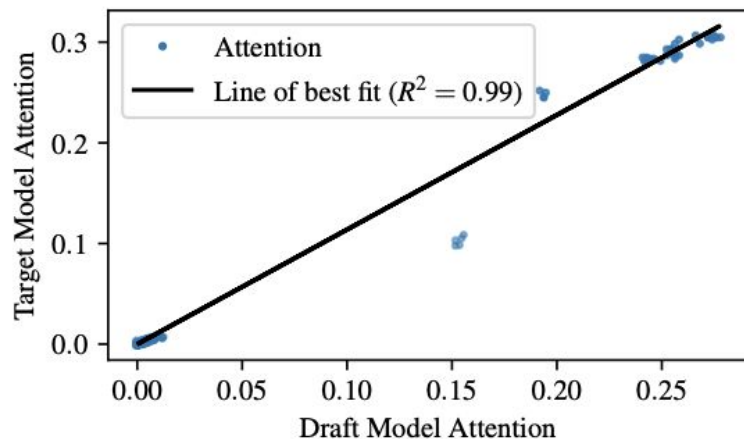
$\triangleright$ Sparse prefill attention

15: cache $\leftarrow K_{i_{\text{cache}}}, V_{i_{\text{cache}}}$

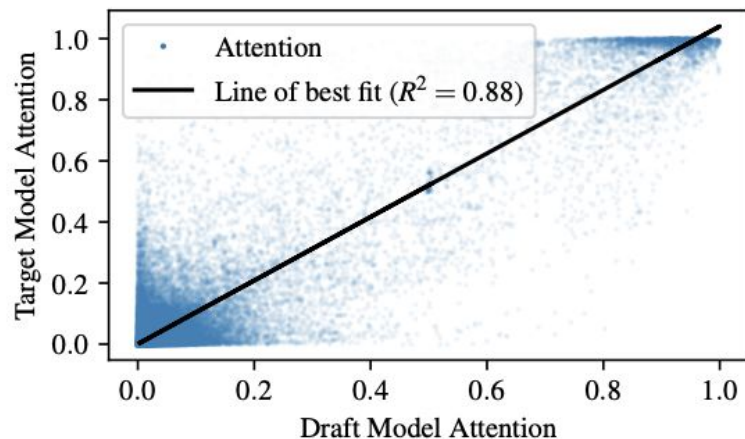
$\triangleright$ KV cache dropping

16: **end for**

SpecPC



(a) Mean aggregation



(b) Max aggregation

Figure 4: Correlation of attention activations between Llama-3.2-1B (draft) and Llama-3.1-8B (target) on 45 RULER (16K) [21] examples (five per each of nine tasks). Attention maps are aggregated across layers, heads, and queries using mean or max per key. Each point plots a key’s activation in the draft versus target model. Both aggregation methods ((a) mean, (b) max) reveal strong correlations.

Algorithm 2 SpecPC

1: Input:

Draft attention tensor $A \in \mathbb{R}^{n_{\text{layer}} \times n_{\text{head}} \times (n_{\text{in}} + n_{\text{lookahead}} - 1) \times n_{\text{in}}}$

Parameters: Window size n_{window} , Kernel size k , Number of neighbors n_{neighbor} ,

Number of selected tokens C_{max} , Number of skipped layers l_{skip}

2: $m \leftarrow n_{\text{in}} - n_{\text{window}}$

3: $A \leftarrow A_{l_{\text{skip}}:, :, m:, :m}$

▷ Skip layers and only consider window queries and non-window keys

4: **for** $j \in \{1, 2, \dots, n_{\text{window}}\}$ **do**

5: $A_{\dots, j, :} \leftarrow \frac{j}{n_{\text{window}}} A_{\dots, j, :}$

▷ Assign more weight to later tokens

6: **end for**

7: $s \leftarrow \text{MaxReduce}(A)$

▷ $\mathbb{R}^{n_1 \times n_2 \times n_3 \times n_4} \rightarrow \mathbb{R}^{n_4}$

8: $s \leftarrow \text{AvgPool1D}(s, k)$

▷ Smooth attention

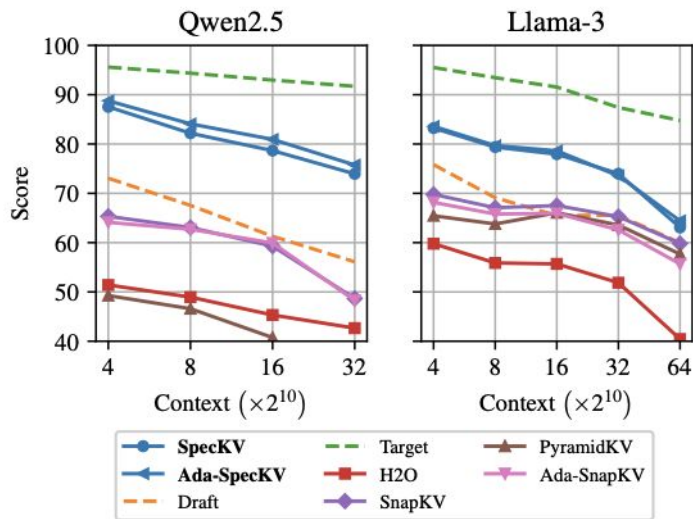
9: $s \leftarrow \text{MaxPool1D}(s, n_{\text{neighbor}})$

▷ Keep neighbor tokens

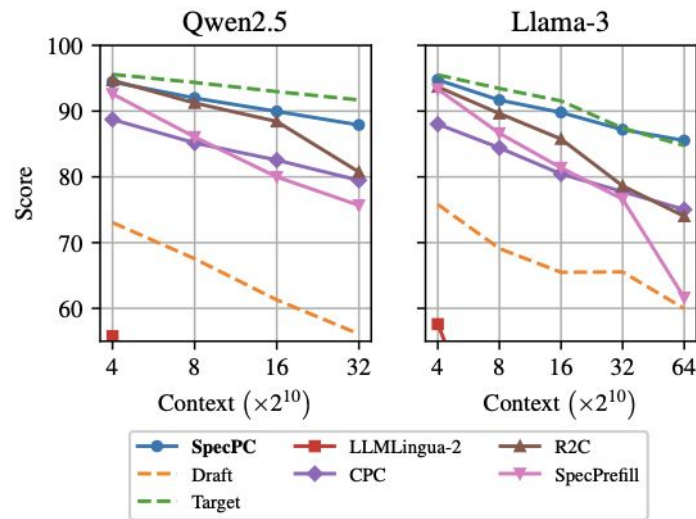
10: $i_{\text{selected}} \leftarrow \text{topk}(s, C_{\text{max}}) \cup \{m + 1, m + 2, \dots, n_{\text{in}}\}$ ▷ Keep most activated tokens and window tokens

11: **return** i_{selected}

Experiments



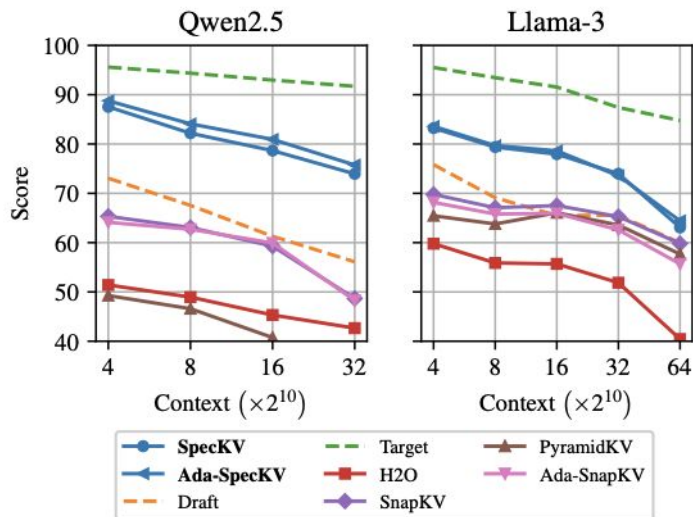
(a) KV dropping ($C_{\max} = 256$)



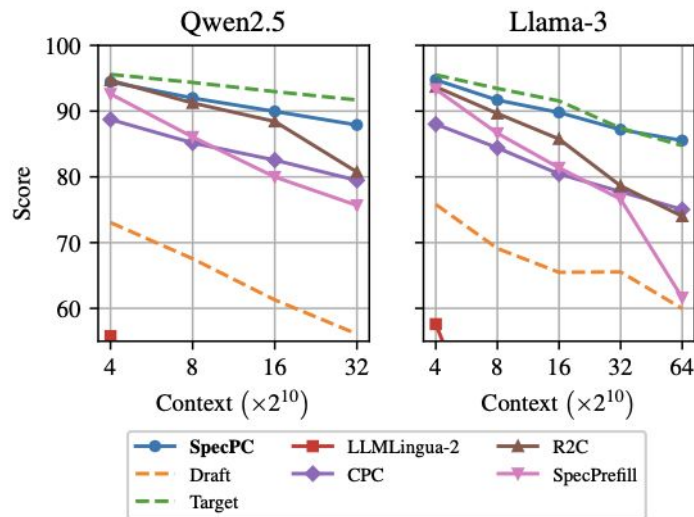
(b) Prompt compression ($C_{\max} = 1024$)

Figure 5: Performance of SpecKV and SpecPC on RULER compared to baselines. Both methods consistently outperform all baselines across sequence lengths. Notably, SpecPC closely matches the target model, maintaining strong results at longer contexts. Low-performing methods are omitted.

Experiments



(a) KV dropping ($C_{\max} = 256$)



(b) Prompt compression ($C_{\max} = 1024$)

Figure 5: Performance of SpecKV and SpecPC on RULER compared to baselines. Both methods consistently outperform all baselines across sequence lengths. Notably, SpecPC closely matches the target model, maintaining strong results at longer contexts. Low-performing methods are omitted.

Experiments

Table 3: LongBench performance with Qwen2.5-0.5B (draft) and Qwen2.5-14B (target).

	$C_{\max}$	Method	Single-doc QA	Multi-doc QA	Summary	Few-shot Learning	Code Completion	All
Dense	–	Draft	21.04	24.4	21.23	54.07	33.39	30.64
		Target	53.19	42.83	24.99	65.34	51.75	47.33
KV	256	StreamingLLM	38.66	24.13	19.10	49.75	35.23	33.24
		H2O	46.98	29.66	19.82	50.88	47.86	38.41
		SnapKV	49.07	33.19	19.49	54.49	47.34	40.25
		PyramidKV	47.07	31.84	18.32	54.81	43.26	38.76
		Ada-SnapKV	50.92	34.57	19.63	55.07	49.58	41.41
		SpecKV	<u>51.07</u>	<u>38.76</u>	<u>23.20</u>	<u>59.68</u>	<u>49.58</u>	<u>44.09</u>
		Ada-SpecKV	51.92	39.38	24.90	61.05	53.43	45.61
PC	1024	LLMLingua-2	27.10	23.74	22.57	35.85	37.62	28.79
		CPC	42.76	36.05	22.58	48.38	35.57	37.17
		R2C	<u>46.43</u>	<u>37.42</u>	22.64	47.21	29.48	37.15
		SpecPrefill	46.21	37.21	<u>22.82</u>	62.18	48.34	<u>43.00</u>
		SpecPC	47.70	38.30	23.30	<u>59.74</u>	52.52	43.73

Table 4: LongBench performance with Llama-3.2-1B (draft) and Llama-3.1-8B (target).

	$C_{\max}$	Method	Single-doc QA	Multi-doc QA	Summary	Few-shot Learning	Code Completion	All
Dense	–	Draft	28.08	27.27	25.65	60.16	31.11	34.69
		Target	45.85	43.79	28.68	66.65	50.46	46.84
KV	256	StreamingLLM	38.69	27.12	21.64	50.75	34.74	34.58
		H2O	43.54	36.81	22.62	55.64	47.81	40.82
		SnapKV	43.79	37.31	21.96	56.29	47.34	40.91
		PyramidKV	43.62	37.79	21.75	55.15	46.32	40.54
		Ada-SnapKV	44.04	37.86	22.34	59.95	50.39	42.38
		SpecKV	43.23	<u>39.73</u>	<u>24.43</u>	60.90	<u>51.09</u>	43.36
		Ada-SpecKV	42.40	40.73	25.74	57.94	52.51	<u>43.25</u>
PC	1024	LLMLingua-2	29.61	24.83	23.43	24.18	40.66	27.68
		CPC	35.67	36.61	25.26	34.01	43.58	34.42
		R2C	38.41	39.07	25.28	43.26	43.99	37.58
		SpecPrefill	<u>41.51</u>	39.38	25.82	<u>63.51</u>	44.68	<u>42.86</u>
		SpecPC	44.83	39.94	25.85	63.70	44.82	43.76